

Simulateur du Microprocesseur Motorola 6809



Encadré par :

Mr. BENALLA Hicham

Réalisé par :

IMZILNE KAWTAR

ID-IDAR HAJAR

ELMAAZOUZI FATNA

OBJECTIF DU PROJET

Objectif Principal

Le projet vise à développer un **simulateur pédagogique complet du microprocesseur Motorola 6809** permettant :

- **L'apprentissage pratique** de la programmation en langage assembleur
- **La visualisation en temps réel** de l'état interne du processeur
- **Le débogage interactif** de programmes assembleur
- **La compréhension des mécanismes** de fonctionnement d'un CPU 8 bits

Objectifs Pédagogiques

Comprendre l'architecture d'un microprocesseur

- Organisation de la mémoire (RAM/ROM)
- Rôle des registres internes
- Fonctionnement des flags de condition

Maîtriser le langage assembleur

- Syntaxe des instructions
- Modes d'adressage
- Gestion de la mémoire

Développer des compétences en débogage

- Exécution pas à pas
- Suivi de l'état des registres
- Analyse du contenu mémoire

Les classes utilisés

❖ Classe Main

Rôle : La classe Main constitue la **classe principale** du simulateur du microprocesseur **Motorola 6809**. Elle assure l'initialisation de l'application, la création de la fenêtre principale et la coordination entre les différentes composantes du simulateur (CPU, RAM, ROM, Programme et Assembleur).

Objectif

- Lancer et initialiser l'exécution globale du simulateur.

- Créer et afficher la fenêtre principale de l'application.
- Centraliser l'accès aux différentes fenêtres du simulateur (CPU, RAM, ROM, Programme, Assembleur).
- Fournir une interface utilisateur permettant d'afficher ou masquer les différentes composantes.

❖ **Classe Assembleur**

Rôle : La classe **Assembleur** assure l'édition, l'analyse et l'exécution des programmes écrits en langage assembleur du microprocesseur **Motorola 6809**. Elle constitue l'interface permettant à l'utilisateur de saisir les instructions assembleur et de lancer leur exécution, soit de manière continue, soit pas à pas.

Objectif

- Permettre la **saisie et l'édition** d'un programme en langage assembleur 6809.

- Analyser les instructions assembleur et identifier les **étiquettes (labels)** du programme.
- Traduire et exécuter les instructions selon le cycle du processeur (**fetch – opcode – exécution**).
- Offrir deux modes d'exécution :
 - exécution **complète**, exécution **pas à pas**.
- Mettre à jour l'état du **CPU** (PC, registres, flags, instruction courante).
- Détecter et signaler les erreurs présentes dans le programme assembleur.

❖ **Classe CPU**

Rôle : La classe **CPU** représente l'architecture interne du microprocesseur **Motorola 6809**. Elle permet d'afficher en temps réel l'état du processeur, notamment les registres et les indicateurs de condition, afin de suivre l'exécution des instructions

Objectif

- Visualiser l'état global du processeur pendant l'exécution du programme.
- Afficher les registres **8 bits** : A, B et DP.
- Afficher les registres **16 bits** : X, Y, S, U et PC
- Afficher les **8 indicateurs (flags)** du registre de condition (CC).

❖ **Classe RAM**

Rôle : La classe **RAM** représente la mémoire vive du système. Elle permet de stocker les variables et les données temporaires utilisées par le programme pendant son exécution et d'en visualiser le contenu en temps réel.

Objectif

- Simuler une mémoire **RAM de 1024 octets (1 Ko)**.
- Permettre la **lecture** et l'**écriture** des données en mémoire.
- Afficher le contenu de la mémoire RAM en temps réel.
- Faciliter l'observation des modifications mémoire lors de l'exécution des instructions.

❖ Classe ROM

Rôle : La classe ROM représente la mémoire morte du simulateur du microprocesseur Motorola 6809. Elle assure l'affichage et la gestion du contenu de la mémoire programme dans une interface graphique.

Objectif

- Simuler le fonctionnement de la mémoire ROM du microprocesseur Motorola 6809..
- Stocker les instructions machine et les données constantes du programme.
- Afficher les adresses mémoire et leurs valeurs en format hexadécimal.
- Faciliter l'observation et le débogage pendant l'exécution du simulateur.

Les instructions utilisés :

- **ORA/ORB** : OU logique registre et valeur
- **AND** : ET logique registre et valeur
- **ASRA/ASRB** : Décalage arithmétique à droite

- **ABX** : Additionne B à X
- **MUL** : Multiplication $A * B \rightarrow M$
- **DECA/DECB** : Décrémente la valeur du registre
- **INCA/INCB**: Incrémente la valeur du registre
- **CLR** : Met le registre à zéro
- **TFR** : Transfert entre registres
- **SUB** : Soustraction registre ou mémoire
- **ADD** : Addition registre ou mémoire
- **LDA / LDB / LDD / LDX / LDY / LDS / LDU** :
Chargement dans registre
- **STA / STB / STD / STX / STY / STS / STU**:
Stockage du registre en mémoire
- **NOP** : Aucune opération
- **END** : Elle indique la fin du programme source à l'assembleur

Les modes d'adressage utilisés :

- **Immédiat**

- **Direct**
- **Étendu**
- **Inhérent**

Les méthodes utilisés :

Méthodes de la classe ROM :

1.ROM() (constructeur)

- Initialise la fenêtre
- Crée le tableau (JTable)
- Initialise la ROM avec les adresses et la valeur par défaut FF

2.modifierValeurColonne(int rowIndex, String valeur)

- Méthode **clé**
- Modifie le contenu mémoire (ROM)
- Écrit un ou deux octets dans le tableau

3.decimalEnHex(int value)

- Conversion **décimal → hexadécimal**
- Utilisée pour afficher correctement les adresses mémoire

4.ObtenirAdresse(int idrom)

- Retourne l'adresse mémoire correspondant à un index ROM
- Utilisée par le processeur (PC)

5.ved_rom()

- Initialise , recharge la ROM avec des valeurs par défaut

Méthodes de la classe ROM :

- **RAM()** : initialise et affiche la mémoire RAM
- **initialiserRAM()** : réinitialise la RAM
- **modifierValeur(...)** : écrit une valeur en mémoire
- **obtenirValeur(...)** : lit une valeur depuis la mémoire

Méthodes de la classe CPU :

- **setA / getA, setB / getB** : manipulation des accumulateurs
- **setD / getD** : gestion du registre 16 bits D
- **setPC()** : mise à jour du compteur ordinal
- **reinitialiser()** : remise à zéro de tous les registres et flags

Méthodes de la classe Programme:

- **afficher_Instructions(...)** : affiche l'instruction avec l'adresse ROM
- **afficher_Instructions(String instru_pc)** : affiche l'instruction sans adresse
- **highlightLine(int number)** : surligne une ligne donnée dans le JTextArea.

Méthodes de la classe Assembleur:

- **Assembleur()** : constructeur qui initialise la fenêtre, les boutons, la zone de texte et le surlignage.
- **executerProgramme_PasAPas(String asmb)** : exécute le programme assembleur ligne par ligne.
- **executerProgramme(String asmb)** : exécute le programme assembleur entièrement jusqu'à l'instruction END.
- **ved_all()** : réinitialise RAM, ROM, CPU, affichage et compteur d'instructions.

- **terminerProgramme()** : traite la fin du programme, met à jour PC et ROM, et désactive les boutons d'exécution.
- **executer_Instr(String[] mots)** : exécute une instruction en fonction du registre et du type d'opération.
- **instr_org(String adresse)** : initialise l'adresse de départ pour ROM.
- **hexToDecimal(String hexString)** : convertit une chaîne hexadécimale en entier décimal.
- **instr_cmp(char registre, String[] mots)** : exécute l'instruction CMP et met à jour les flags N et Z.
- **instr_or(char registre, String[] mots)** : exécute l'instruction OR et met à jour les flags N, Z, V, C, H.

Le programme possède des méthodes pour exécuter le programme en entier ou pas à pas, gérer la mémoire et le CPU, et chaque instruction est implémentée dans sa propre méthode.

Conclusion :

Ce projet a permis de concevoir un simulateur simple mais fonctionnel pour l'architecture du processeur Motorola 6809. Il intègre les principaux composants d'un ordinateur : la **RAM**, la **ROM**, le **CPU** et un **éditeur de programme assembleur**.

Chaque composant a été modélisé avec une interface graphique interactive, permettant de visualiser l'état des registres, de la mémoire et des instructions exécutées.

Le simulateur offre la possibilité d'exécuter un programme assembleur **pas à pas** ou en **exécution complète**, avec mise à jour automatique de la mémoire et des registres, ainsi que le suivi des flags du CPU. Les instructions assembleur sont implémentées individuellement comme méthodes, ce qui rend le code modulable et facile à maintenir ou à étendre.

Ce travail a renforcé la compréhension de l'architecture interne d'un processeur, des mécanismes de la mémoire et de l'exécution des instructions, tout en appliquant les concepts de programmation orientée objet et d'interfaces

graphiques en Java. Il constitue une base solide pour développer ultérieurement un simulateur plus complet, intégrant des fonctionnalités avancées comme les interruptions, les sous-programmes et la gestion d'entrées/sorties.